Map Windows Event IDs to event_type automatically

We a’re manually interpreting Event IDs. Let’s automate it so your pipeline produces clean fields like:

```
{
 "event_id": 4625,
 "event_type": "login_failed"
}
```

GOAL

Event ID → Human-readable event_type → Used by detection engine

---

WHY THIS MATTERS

Windows logs look like this:

Event ID: 4688

That means nothing to your detection logic unless you map it to: process_execution

This is called **normalization** (critical in SIEM/EDR systems)

BUILD EVENT ID → EVENT TYPE MAP

---

event_mapper.py

```
EVENT_ID_MAP = {
  # Authentication
  4624: "login_success",
  4625: "login_failed",
  4634: "logoff",

  # Process activity
```

```
    4688: "process_created",
    4689: "process_terminated",

    # File access
    4663: "file_access",
    4656: "file_handle_requested",

    # Privilege use
    4672: "admin_logon",

    # Account changes
    4720: "user_created",
    4726: "user_deleted",

    # Policy changes
    4719: "audit_policy_changed",

    # Network-related (limited in default logs)
    5156: "network_connection_allowed",
    5157: "network_connection_blocked"
}
```

CREATE MAPPING FUNCTION

```
def map_event_type(event):
    event_id = event.get("event_id")

    event_type = EVENT_ID_MAP.get(event_id, "unknown")

    event["event_type"] = event_type

    return event
```

APPLY TO MULTIPLE EVENTS

```
def map_all_events(events):
```

```
    return [map_event_type(e) for e in events]
```

INTEGRATE INTO YOUR PIPELINE

Update your `main.py`

```
from event_mapper import map_all_events

# After collecting logs
logs = collect_logs()

# Add this step
logs = map_all_events(logs)
```

BEFORE vs AFTER

Before

```
{
 "event_id": 4625
}
```

After

```
{
 "event_id": 4625,
 "event_type": "login_failed"
}
```

HOW DETECTION IMPROVES

Old detection

if e["event_id"] == 4625:

New detection (cleaner)

if e["event_type"] == "login_failed":

Why this is better

✔ Easier to read
✔ Easier to extend
✔ Works across different log sources

EXTENDING THE MAPPING (ADVANCED)

Add MITRE mapping inline

```
EVENT_ID_MAP = {
  4625: {
     "event_type": "login_failed",
     "mitre": {
        "technique": "T1110",
        "tactic": "Credential Access"
     }
  },
  4688: {
     "event_type": "process_created",
     "mitre": {
        "technique": "T1059",
        "tactic": "Execution"
     }
  }
}
```

Updated function

```
def map_event_type(event):
  mapping = EVENT_ID_MAP.get(event.get("event_id"))

  if mapping:
    event["event_type"] = mapping["event_type"]
    event["mitre"] = mapping.get("mitre")

  else:
    event["event_type"] = "unknown"

  return event
```

ADDING CONTEXT (OPTIONAL BUT POWERFUL)

You can enrich events:

```
if event["event_type"] == "login_failed":
  event["category"] = "authentication"
```

FINAL RESULT

Your logs now look like:

```
{
 "event_id": 4625,
 "event_type": "login_failed",
 "category": "authentication",
 "mitre": {
   "technique": "T1110"
 }
}
```

WHAT I JUST BUILT

✔ Log normalization layer
✔ Detection-ready data model
✔ MITRE-ready enrichment

This is exactly what:

- Splunk
- Elastic Stack
- Microsoft Sentinel

…do internally